Training Run Monitor

Frontend Engineer take-home submission | November 2026

Project and architecture

A responsive monitoring interface for 20 synthetic machine-learning training runs, including loss, learning rate, and gradient norm charts. It helps a reviewer judge whether costly training should continue without relying on misleading smoothing or missing-data assumptions. Built with React, TypeScript (strict), Vite, ECharts (SVG renderer), Vitest, and Playwright. Data generation and analysis run in a Web Worker that returns typed-array columns plus a null mask, so NaN, ±Infinity and `null` survive the transfer. The chart code is lazy-loaded, and search, filters and sort live in the URL.

Live app: https://training-run-monitor-gold.vercel.app/ (Vercel, deployed commit `20654d3`) | **Repository:** https://github.com/rauantuken06/Training-Run-Monitor

Data truthfulness and validation

The dataset includes NaN, positive/negative Infinity, null, out-of-order and duplicate steps, missing intervals, and a 1,000,000-point run with one isolated spike at step 500,123, loss 42.5. The integrity layer retains raw observations, original indices, explicit diagnostics, and line breaks across invalid values or missing steps. `null` means "not recorded"; NaN and ±Infinity mean "recorded but invalid". None is ever plotted as zero: each appears as a labelled marker (for example "NaN ×600") and a written warning.

Viewport-aware, segment-preserving min/max downsampling keeps original coordinates and visible extrema rather than sampling every Nth observation; it was checked against a brute-force reference. ECharts uses separate series for valid segments, a shared zoom range, linear axes, accurate hover semantics, and visible anomaly markers. Conflicting duplicates are disclosed; the display policy selects the last recorded value, while diagnostics retain the alternatives.

Verification evidence

Check	Result / scope
Unit / browser tests (local production build)	564 unit tests passed. E2E: Chrome 86 passed and Edge 86 passed (8 intentional skips each), Firefox 46 passed (1 skip); 0 failed. Lint, typecheck and build pass.
Production smoke tests (live URL, commit `20654d3`; 12 tests in desktop Chrome and Pixel 7 emulation: direct routes, refresh, assets, 1M spike/zoom/hover, invalid-value warnings)	**Candidate:** first 4-worker run 12/12 passed (original log not retained); 4-worker rerun **9 passed, 3 timed out** (log saved); 1-worker run 12/12 passed (log saved). **AI agent:** independent 1-worker run 12/12 passed (log saved). The root cause of the parallel timeouts is **unconfirmed**, and they have not been fixed.
Accessibility / security	axe reported 0 violations (WCAG 2.0–2.2 A/AA) in the tested light/dark states; in Firefox, six rules were skipped on one state. Focus outlines and chart colour contrast are checked by automated tests, and the Tab order was walked through by the AI agent. This is not a manual audit by the candidate. npm audit: 0 production vulnerabilities, 7 moderate dev-tool findings.

Performance before / after

Metric (Chrome, CPU 4×)	Before	After
1M run: total main-thread blocking	3,638 ms	1,023 ms (−72%)
1M run: longest task	1,824 ms	332 ms (−82%)
First charts painted	4,348 ms	2,530 ms (−42%)*
New zoom viewport / revisited viewport	504 / 360 ms	440 / 144 ms
Hover input to next frame (median)	32 ms	31 ms
Runs-list JavaScript downloaded	950 kB	371 kB (−61%)

* Chrome's 4× CPU throttling does not slow the Web Worker equally, so end-to-end loading gains overstate the expected gains on a genuinely slower CPU. About 3.8 s is an estimate for that case, not a measurement. Measurements are medians of five runs; traces are in `docs/performance/{baseline,after}/traces/`.

Findings, decision, and limitations

Submission report | Technical conclusions

Silent failures: what looked correct but was not

Observed defect	Resolution
Locale-dependent zoom text was misread ("5.000" became step 5)	Accept unambiguous whole-step formats and explain rejected input.
Requested viewport silently shifted	Trim only overlapping bounds; reject wholly empty ranges.
Tooltip labelled a nearby sample as the cursor step	Distinguish cursor range from nearest plotted observation.
Different axis ticks displayed identical labels	Adapt numeric precision to the actual viewport.
Fast typing dropped search characters	Maintain local input state while synchronizing with URL changes.
Axis contrast insufficient; hung Worker stayed loading	Improve contrast; add a timed, visible failure and retry.

Each defect was reproduced first and is now covered by a regression test.

SHIP verdict and next priorities

SHIP. The public deployment passed all 12 production smoke tests in the first parallel run and in both sequential runs, one by the candidate and one by the AI agent. One parallel rerun had 3 timeouts; their root cause is unconfirmed. In the passing runs, the key 1M-point spike remained visible and hoverable after zoom, and invalid values were shown as labelled warnings, not silently converted to zero. The product is suitable as a tested take-home demonstration, not a certified production ML observability system.

Next priorities: further reduce first-time zoom latency (~440 ms under 4× CPU throttling); investigate the parallel production smoke-test timeouts; test with a real screen reader and slower physical devices; and establish real-backend freshness and request-race guarantees before using this with live training. Tests cannot guarantee that extrema-preserving reduction retains every oscillation or minor secondary event.

Memory and measurement limitations

With the 1M-point run open, Windows process counters showed about 123–128 MB settled private memory and 223–225 MB peak during loading. Main-thread JS heap was about 74.9 MB. These are not Chrome Task Manager memory-footprint measurements. Browser profiling used automated Chrome traces with CPU throttling at 4×; the worker was not throttled to the same degree. Production smoke tests ran only in Chrome (desktop and Pixel 7 emulation); Edge and Firefox were tested only against the local production build. Safari, real devices, and a manual screen-reader session were not verified.

AI-assisted workflow and verification

- **ChatGPT:** planning and review. It helped structure the work into verifiable milestones and reviewed the reported results.
- **Claude Code** (Anthropic's coding agent in VS Code): implementation and verification. It wrote most of the code, tests and documentation, ran lint, typecheck, unit, e2e and build, recorded the performance traces and memory measurements, ran the accessibility checks described above, and independently re-ran the production smoke test (12/12, 1 worker).
- **Candidate:** wrote the task specifications, set the Git workflow (work on `main`, commit and push only with approval), made the commits and pushes, merged the pull requests, deployed to Vercel, confirmed the deployed commit `20654d3`, and ran the three production smoke runs listed above.

Suggestions were evaluated against test evidence; forced dependency upgrades were avoided, and detected defects were addressed with regression checks. This does not imply that every code path was manually inspected, or that the candidate repeated every automated check without assistance.

Reflection

A key lesson was that a chart can look reasonable yet communicate the wrong step, hide a gap, or make different values appear identical. Browser-level checks uncovered errors missed by unit tests, while profiling showed that removing main-thread blocking does not necessarily mean equal speedups on slower machines. The remaining concerns are interaction latency on constrained hardware and accessibility beyond automated audits.

Evidence and reproducibility

Repository evidence: `docs/test-evidence.md`, `docs/qa/logs/` (including `production-smoke-vercel.log` for the failed parallel rerun, `production-smoke-sequential.log`, and `production-smoke-agent-sequential.log`), `docs/performance-report.md`, `docs/performance/{baseline,after}/traces/`, `docs/silent-failures-audit.md`, and `docs/final-qa-report.md`. Every result in this report is copied from those logs and reports. The only exception is the first parallel production run, which is reported by the candidate and has no saved log.